

COMM114: Generative AI - Recurrent Neural Networks (RNNs)

Guoqiang Zhang

November 2025

Machine Learning for Modeling Sequences

- When applying machine learning to sequences, we often want to turn an input sequence into an output sequence that lives in a different domain
 - Example 1: To turn a sequence of audio signals into a sequence of words.
 - Example 2: To turn a sentence in one language into a sequence in another language while preserving the meaning.
- When there is only a single sequence, we are interested in training a machine learning model to predict the next term in the sequence based the past terms.
 - In the training process, the target output sequence is the input sequence with an advance of 1 step.
 - Roughly speaking, ChatGPT is trained by predicting the next word in a document based on the previous words.
 - It reduces to unsupervised learning.

Modeling Sequences via Probability

- Suppose the joint distribution of the terms in sequences is characterized by $P_{data}(\{Y_i\}_{i=1}^n)$.
 - Assumption: Each Y_i is a discrete random variable, taking values from a discrete set Ω .
 - For example, for natural language processing, the discrete set includes all the possible words or subwords.
- It is known from probability theory that the joint distribution can be decomposed as

$$P_{data}(\{Y_i\}_{i=1}^n) = P_{data}(Y_1) \prod_{i=2}^n P_{data}(Y_i | \{Y_{k < i}\})$$

where $\{Y_{k < i}\}$ includes all the random variables up to index i .

Remark: For simplicity, we only consider discrete random variables. One can also extend the analysis to continuous case.

Auto-Regressive (AR) Models

- An AR model $P_{\theta}(\{Y_i\}_{i=1}^n)$ attempts to learn the conditional distribution $P_{data}(Y_i|\{Y_{k<i}\})$ for all index i .
- Once an AR model is trained, it can be used to generate new sequences in a recurrent manner, one term per timestep according to the learned conditional probability.
- In general, those popular AR models in the literature are mainly different in their architectures.

Popular AR models in the literature:

- Recurrent neural networks
- Transformer-based AR models
 - They dominate natural language processing (NLP)
 - Transformer can also be used in diffusion and flow-matching models. Generally speaking, Transformer is not an AR model in diffusion and flow-matching models because each token uses all other tokens for updating.
- State space models (variants of RNN models)

Recurrent Neural networks (RNNs)

- A RNN model introduces hidden states to store information about the past efficiently.
- The dimensionality of the hidden state can be adjusted depending on the complexity of the data sequences.

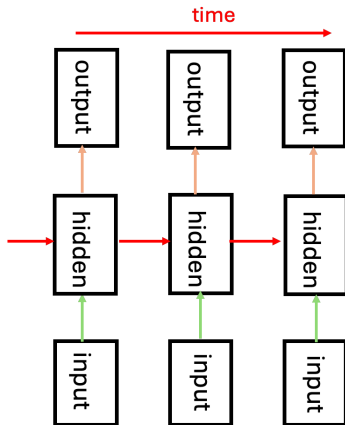
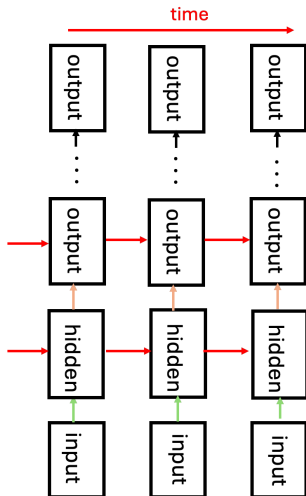


Figure: A RNN with a single hidden layer

RNNs (cont.)

- If necessary, the RNN model can be built with multiple hidden layers to enhance its representational capacity.



A simple RNN architecture

- The simple RNN model has an internal state that is updated as a sequence is processed one term after another in a casual manner.

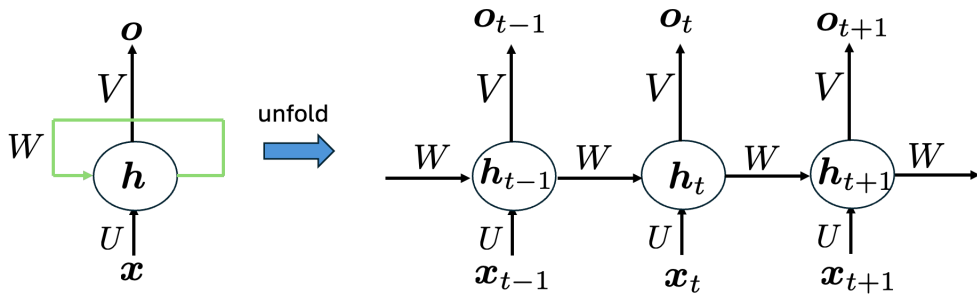


Figure: A RNN with a single hidden layer

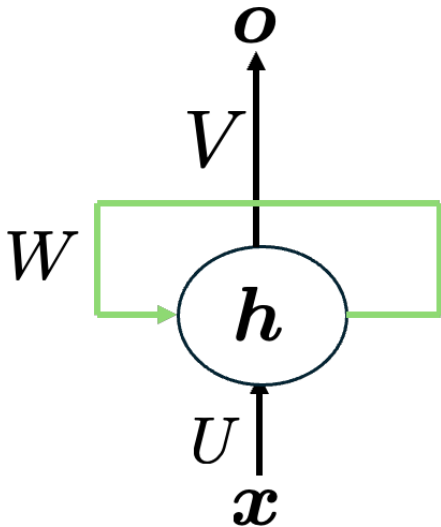
A simple RNN architecture (cont.)

- Update expression:

$$\mathbf{h}_t = \text{activation}(U\mathbf{x}_t + W\mathbf{h}_{t-1})$$

$$\mathbf{o}_t = \text{softmax}(V\mathbf{h}_t)$$

- The activation function can be, for example, ReLU, tanh, sigmoid, and so on.
- Suppose $\mathbf{x}_t, \mathbf{o}_t \in \mathbb{R}^d$, and $\mathbf{h}_t \in \mathbb{R}^k$.
- $W \in \mathbb{R}^{k \times k}$ is always a square matrix.



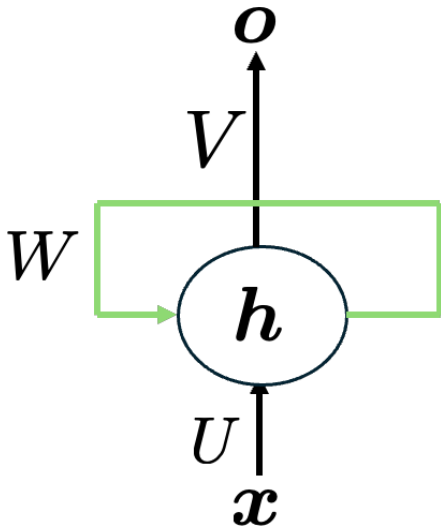
A simple RNN architecture (cont.)

- Alternative update expression:

$$\mathbf{h}_t = \alpha \mathbf{h}_{t-1} + (1 - \alpha) \text{activation}(U \mathbf{x}_t + W \mathbf{h}_{t-1})$$

$$\mathbf{o}_t = \text{softmax}(V \mathbf{h}_t)$$

- The above expression is designed to better preserve the information of the past hidden states when computing the current one.



Exploiting RNN for learning the conditional distribution

- Let $\mathbf{y} = [y_1, y_2, \dots, y_n]$ be a particular sequence drawing from the data distribution $P_{data}(\{Y_i\}_{i=1}^n)$.
- Converting each y_i into a one-hot vector $\mathbf{x}_i \in \mathbb{R}^d$.
 - Example: suppose the discrete set $\Omega = \{\text{"red"}, \text{"blue"}, \dots, \text{"green"}\}$ has d elements.
 - $\mathbf{x}_i = \text{one-hot}(y_i = \text{"red"}) = [1, 0, \dots, 0]$
 - $\mathbf{x}_i = \text{one-hot}(y_i = \text{"blue"}) = [0, 1, \dots, 0]$.
 - $\mathbf{x}_i = \text{one-hot}(y_i = \text{"green"}) = [0, 0, \dots, 1]$.

Exploiting RNN for learning the conditional distribution (cont.)

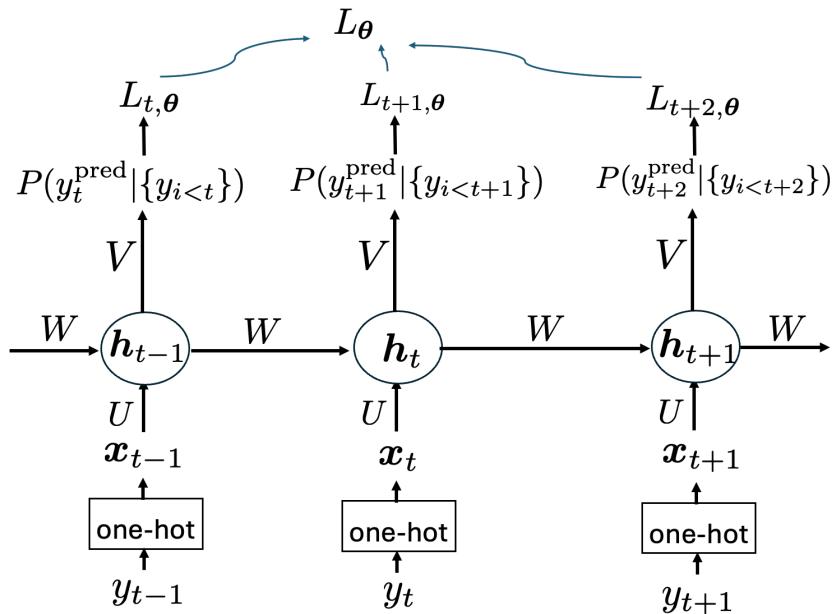
- Let $\mathbf{y} = [y_1, y_2, \dots, y_n]$ be a particular sequence drawing from the data distribution $P_{data}(\{Y_i\}_{i=1}^n)$.
- Converting each y_i into a one-hot vector $\mathbf{x}_i \in \mathbb{R}^d$.
- At each timestep t , feed $\mathbf{x}_t = \text{one-hot}(y_t)$ into the RNN model and compute the loss function

$$L_{t,\theta} = \text{cross-entropy} \left(P \left(y_{t+1}^{\text{pred}} | \{y_{i < t}\} \right), P(y_{t+1}) \right),$$

- $\theta = (U, W, V)$
- $\{y_{i < t}\}$ includes all the samples up to index t . $\{y_{i < t}\} = \{y_1, y_2, \dots, y_{t-1}\}$.
- Update the model parameters by minimizing the overall loss function via back-propagation:

$$\arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n L_{i,\theta}$$

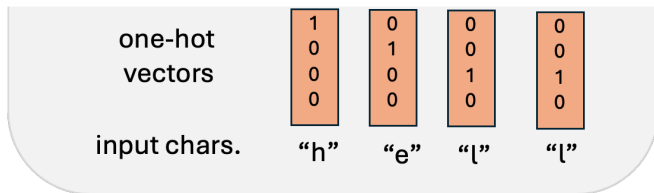
Exploiting RNN for learning the conditional distribution (cont.)



Exploiting RNN for learning the conditional distribution (cont.)

Example

- Character-level Language Model:
 - Vocabulary: [h, e, l, o]
 - A training sequence: "hello"



Exploiting RNN for learning the conditional distribution (cont.)

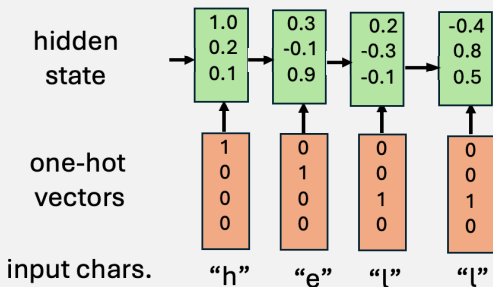
Example

- Character-level Language

Model:

- Vocabulary: [h, e, l, o]
- A training sequence: "hello"

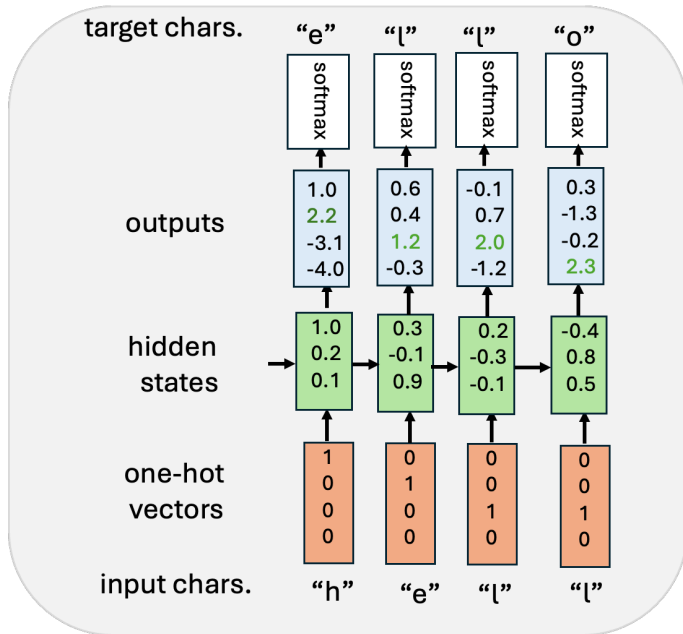
$$\mathbf{h}_t = \tanh(W\mathbf{h}_{t-1} + U\mathbf{x}_t)$$



Exploiting RNN for learning the conditional distribution (cont.)

Example

- Character-level Language Model:
 - Vocabulary: [h, e, l, o]
 - A training sequence: "hello"



Exploiting RNN for learning the conditional distribution (cont.)

Computational Complexities:

- For a sequence of n terms, the RNN model needs to be unfolded n times to compute the overall loss function.
- The computational complexity of the forward pass in a RNN is of order $O(n)$.
- The computational complexity of the backpropagation in a RNN is also of order $O(n)$ from the last term in the sequence to the first one.

Exploiting RNN for learning the conditional distribution (cont.)

Challenges when training an RNN

- Instable gradient computation due to the fact that gradients are multiplied over many time steps during training:
 - **Vanishing Gradients:** When the gradients become very small, the network has difficulty learning from distant past terms in the sequence, as the "loss functional value" becomes too weak.
 - **Exploding Gradients:** If the gradients grow too large, it can cause massive instability in the training process.

Sequence generation from a pretrained RNN

Suppose a RNN model is well trained. How to generate a new sequence from the pretrained RNN?

- Operations at timestep 1:
 - Firstly, select y_1 from the discrete set Ω .
 - Secondly, take y_1 as input the RNN model to obtain $P(y_2^{\text{pred}}|y_1)$.
 - Thirdly, generate a new sample $\hat{y}_2$ according to $P(y_2^{\text{pred}}|y_1)$.
- Operations at timestep 2:
 - Firstly, take $\hat{y}_2$ as input the RNN model to obtain $P(y_3^{\text{pred}}|y_1, \hat{y}_2)$.
 - Secondly, generate a new sample $\hat{y}_3$ according to $P(y_3^{\text{pred}}|y_1, \hat{y}_2)$.
- Repeat the above generation process to obtain $\{y_1, \hat{y}_2, \dots, \hat{y}_n\}$.

Sequence generation from a pretrained RNN (cont.)

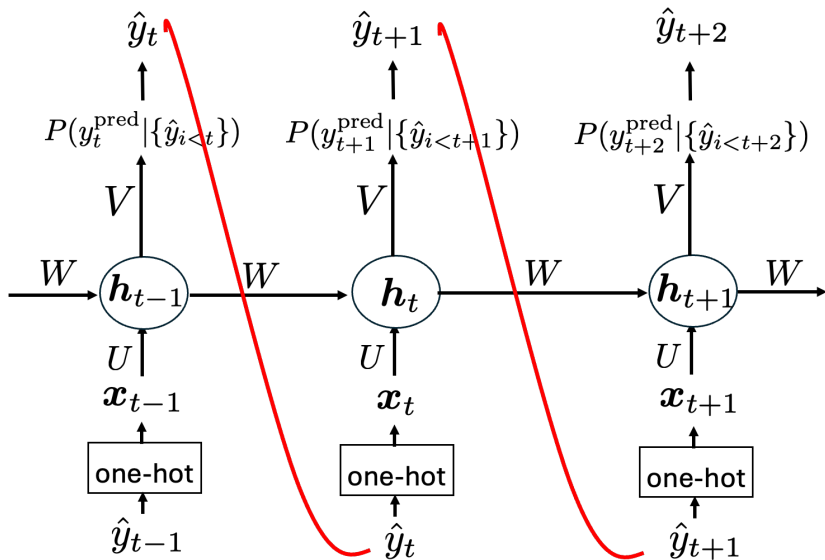
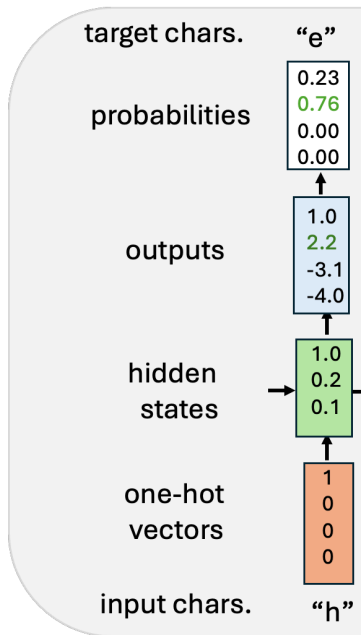


Figure: Diagram for sequence generation from a RNN.

Sequence generation from a pretrained RNN (cont.)

Example

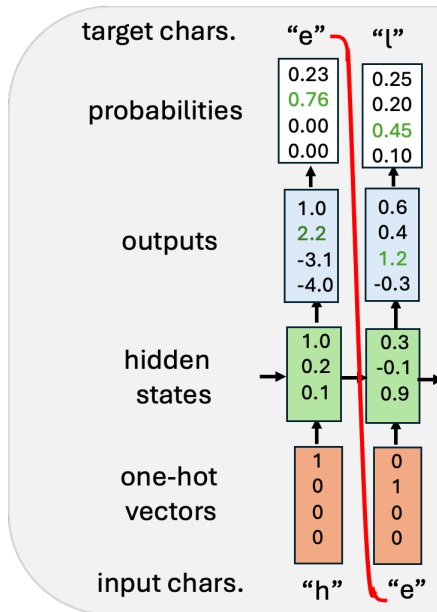
- Character-level Language Model:
 - Vocabulary: [h, e, l, o]
 - At inference stage, generate characters one at a time, feed back to the model



Sequence generation from a pretrained RNN (cont.)

Example

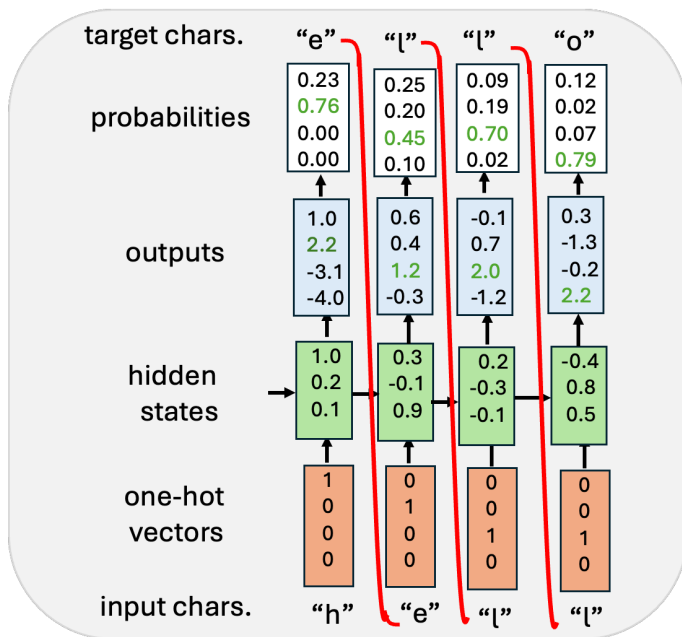
- Character-level Language Model:
 - Vocabulary: [h, e, l, o]
 - At inference stage, generate characters one at a time, feed back to the model



Sequence generation from a pretrained RNN (cont.)

Example

- Character-level Language Model:
 - Vocabulary: [h, e, l, o]
 - At inference stage, generate characters one at a time, feed back to the model



min-char-rnn.py gist: [code-link](#)

```
5 import numpy as np
6
7 # data I/O
8 data = open('input.txt', 'r').read() # should be simple plain text file
9 chars = list(set(data))
10 data_size, vocab_size = len(data), len(chars)
11 print('data has %d unique, ' % (data_size, vocab_size))
12 char_to_ix = { ch:i for i, ch in enumerate(chars) }
13 ix_to_char = { ix:ch for i, ch in enumerate(chars) }
14
15 # hyperparameters
16 hidden_size = 100 # size of hidden layer of neurons
17 seq_length = 25 # number of steps to unroll the RNN for
18 learning_rate = 1e-1
19
20 # model parameters
21 Wxh = np.random.randn(hidden_size, vocab_size)*0.01 # input to hidden
22 Whh = np.random.randn(hidden_size, hidden_size)*0.01 # hidden to hidden
23 Why = np.random.randn(vocab_size, hidden_size)*0.01 # hidden to output
24 bh = np.zeros((hidden_size, 1)) # hidden bias
25 by = np.zeros((vocab_size, 1)) # output bias
26
27 def lossFun(inputs, targets, hprev):
28     """
29     inputs, targets are both list of integers.
30     hprev is Hx1 array of initial hidden state
31     returns the loss, gradients on model parameters, and last hidden state
32     """
33     xs, hs, ys, ps = {}, {}, {}, {}
34     hs[-1] = np.copy(hprev)
35     loss = 0
36     # forward pass
37     for t in xrange(len(inputs)):
38         xs[t] = np.zeros((vocab_size, 1)) # encode in 1-of-k representation
39         xs[t][inputs[t]] = 1
40         hs[t] = np.tanh(np.dot(Wxh, xs[t]) + np.dot(Whh, hs[t-1]) + bh) # hidden state
41         ys[t] = np.dot(Why, hs[t]) + by # unnormalized log probabilities for next chars
42         ps[t] = np.exp(ys[t]) / np.sum(np.exp(ys[t])) # probabilities for next chars
43         loss += -np.log(ps[t][targets[t], 0]) # softmax (cross-entropy loss)
44     # backward pass: compute gradients going backwards
45     dWxh, dWhh, dWhy = np.zeros_like(Wxh), np.zeros_like(Whh), np.zeros_like(Why)
46     dbh, dby = np.zeros_like(bh), np.zeros_like(by)
47     dhnext = np.zeros_like(hs[0])
48     for t in reversed(xrange(len(inputs))):
49         dy = np.copy(ps[t])
50         dy[targets[t]] -= 1 # backprop into y. see http://cs231n.github.io/neural-networks-4
51         dWhy += np.dot(dy, hs[t].T)
52         dby += dy
53         dh = np.dot(Why.T, dy) + dhnext # backprop into h
54         dhraw = (1 - hs[t]**2) * dh # backprop through tanh nonlinearity
55         dbh += dhraw
56         dWxh += np.dot(dhraw, xs[t].T)
57         dWhh += np.dot(dhraw, hs[t-1].T)
58         dhnext = np.dot(Whh.T, dhraw)
59     for dparam in [dWxh, dWhh, dWhy, dbh, dby, hs[len(inputs)-1]]:
60         np.clip(dparam, -5, 5, out=dparam) # clip to mitigate exploding gradients
61     return loss, dWxh, dWhh, dWhy, dbh, dby, hs[len(inputs)-1]
```

```
62
63 def sample(h, seed_ix, n):
64     """
65     sample a sequence of integers from the model
66     h is memory state, seed_ix is seed letter for first time step
67     """
68     x = np.zeros((vocab_size, 1))
69     x[seed_ix] = 1
70     ixes = []
71     for t in xrange(n):
72         h = np.tanh(np.dot(Wxh, x) + np.dot(Whh, h) + bh)
73         y = np.dot(Why, h) + by
74         p = np.exp(y) / np.sum(np.exp(y))
75         ix = np.random.choice(range(vocab_size), p=p.ravel())
76         x = np.zeros((vocab_size, 1))
77         x[ix] = 1
78         ixes.append(ix)
79     return ixes
80
81 n, p = 0, 0
82 mWxh, mWhh, mWhy = np.zeros_like(Wxh), np.zeros_like(Whh), np.zeros_like(Why)
83 mbh, mby = np.zeros_like(bh), np.zeros_like(by) # memory variables for Adagrad
84 smooth_loss = -np.log(1.0/vocab_size)*seq_length # loss at iteration 0
85 while True:
86     # prepare inputs (we're sweeping from left to right in steps seq_length long)
87     if p+seq_length+1 >= len(data) or n == 0:
88         hprev = np.zeros((hidden_size, 1)) # reset RNN memory
89         p = 0 # go from start of data
90         inputs = [char_to_ix[ch] for ch in data[p:p+seq_length]]
91         targets = [char_to_ix[ch] for ch in data[p+1:p+seq_length+1]]
92
93     # sample from the model now and then
94     if n % 100 == 0:
95         sample_ix = sample(hprev, inputs[0], 200)
96         txt = ''.join(ix_to_char[ix] for ix in sample_ix)
97         print '----\n %s \n----' % (txt, )
98
99     # forward seq_length characters through the net and fetch gradient
100     loss, dWxh, dWhh, dWhy, dbh, dby, hprev = lossFun(inputs, targets, hprev)
101     smooth_loss = smooth_loss * 0.999 + loss * 0.001
102     if n % 100 == 0: print 'iter %d, loss: %f' % (n, smooth_loss) # print progress
103
104     # perform parameter update with Adagrad
105     for param, dparam, mem in zip([Wxh, Whh, Why, bh, by],
106                                   [dWxh, dWhh, dWhy, dbh, dby],
107                                   [mWxh, mWhh, mWhy, mbh, mby]):
108         mem += dparam * dparam
109         param += -learning_rate * dparam / np.sqrt(mem + 1e-8) # adagrad update
110
111     p += seq_length # move data pointer
112     n += 1 # iteration counter
```

Generated Texts Across Training Process

tyntd-iafhatawiaoihrdemot lytdws e ,tfti, astai f ogoh eoase rrranbyne 'nhthnee e
plia tklrgrd t o idoe ns,smtt h ne etie h,hregtrs nigtike,aoaenns lng

↓
train more

"Tmont thithey" fomesscerliund
Keushey. Thom here
sheulke, anmerenith ol sivh I lalterthend Bleipile shuw y fil on aseterlome
coaniogennc Phe lism thond hon at. MeiDimorotion in ther thize."

↓
train more

Aftair fall unsuch that the hall for Prince Velzonski's that me of
her hearly, and behs to so arwage fiving were to it beloge, pavu say falling misfort
how, and Gogition is so overelical and ofter.

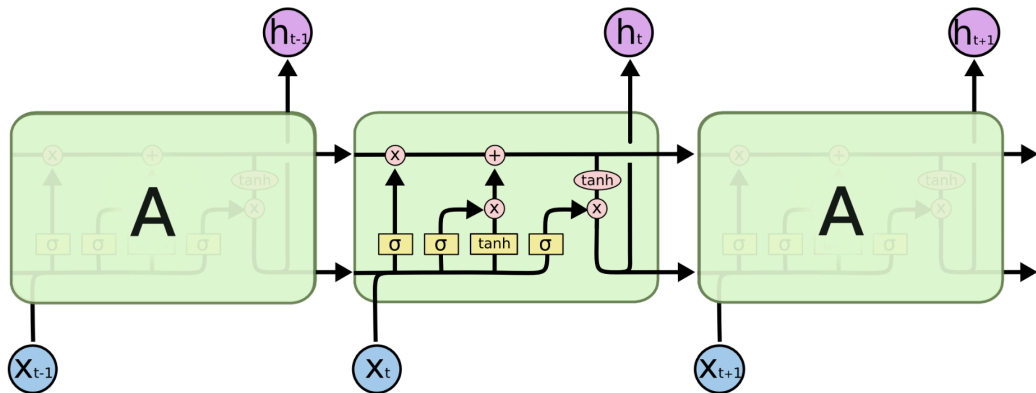
↓
train more

"Why do what that day," replied Natasha, and wishing to himself the fact the
princess, Princess Mary was easier, fed in had oftened him.
Pierre aking his soul came to the packs and drove up his father-in-law women.

Long-Short Term Memory (LSTM)

LSTM, or Long Short-Term Memory, is a type of Recurrent Neural Network (RNN) specifically designed to learn and retain information from sequences of data over long time intervals, overcoming the vanishing gradient problem that plagues traditional RNNs. It achieves this through internal "gates" (forget, input, and output) that control the flow of information, allowing the network to remember important details while discarding irrelevant ones, making it highly effective for tasks like natural language processing and time series analysis. (from google)

LSTM (cont.)



References

1. Tutorial by Fei-Fei Li, Jiajun Wu, and Ruohan Gao

Questions?